Competitive Esports Network: A Relational Approach to Tournament Management Phase III

Wyatt Fischer
fisc7842@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Nastia Kossiak
koss0519@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Brandon Lunney
lunn2076@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, US

Abstract

An update on the E-Sports Competitive Network is presented. A tool that is designed to showcase e-sports players, teams, and tournaments through a graphic interface while utilizing a relational database. The description of the current working prototype is given as well as changes to the entity relational model and tech stack since the previous report. Showcases of all current web pages of the project are detailed with mass database population and manual insertion. Implementation details of the three key queries utilized by the tool are described. Lastly, a reflection on the timeline since project start.

CCS Concepts

• **Relational Databases**; • **Computer Games**;

Keywords

Esports, Relational Databases, MySQL, Cytoscape, Network Visualization, Python

ACM Reference Format:
Wyatt Fischer, Nastia Kossiak, and Brandon Lunney. 2024. Competitive Esports Network: A Relational Approach to Tournament Management Phase III. In . ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

E-Sports is one of the fastest growing media in recent years. The biggest e-sports events can even receive upwards of 5.5 million views. With a wide range of games, including titles such as *League of Legends*, *Counter-Strike*, and *Dota 2*, the industry has developed a vast ecosystem of professional teams and large-scale tournaments. However, unlike it's real life counterpart e-sports does not have near the amount of statistical analysis tools as regular sports. These tools allow coaches, fans, and researchers to better understand performance trends, strategic strengths, and the relationships between teams across seasons. This was the motivation behind the *E-Sports Competitive Network*. For this project we will be implementing a relational database to facilitate statistical analysis of e-sports players and teams across several seasons and games. We will then utilizing that database to visually communicate the information in a graph format.

The requirements of this project are we facilitate multiple types of nodes, in this instance this is players, teams, and the tournaments themselves. This project will also allow for several edges types, such that teammates, played in, and played for. We also allow for multi-graph support, specifically the ability for a node to belong to several graphs and by extension edges to multiple graphs as well. There will also feature a minimum set of queries available, such as an h-index filter or for our purposes a win rate filter for the teams and players. As well as a Co-Author network which for this system will be a teammate network. Lastly, there will be a Q1 journal influence network. However, for our purposes this will be a Q1 tournament influence network, where the Q1-Q4 will be determined based on which quartile the prize pool falls into. Then we will find all teams and by extension the players that participated in that tournament.

2 Prototype Description

The E-Sports Competitive Network prototype is designed to address the lack of advanced statistical analysis tools in competitive gaming. This prototype introduces a relational database system that models graph connections between players, teams, and tournaments across multiple games. The prototype implements three key analytical queries. First, a teammate influence network that constructs a teammate graph showing how players are connected. Second, an h-index filer, identifies consistently high-performing teams based on either participation or number of wins. Lastly, a Q1-Journal Influence Network that identifies the most impactful tournaments based on prize pool and connects them to the participating teams. The prototype also allows for database population with a custom web scrapping tool as well as several web pages to manually insert information.

3 Updated Entity Relation Model

In previous reports Figure 1 depicts the entity relation diagram. Which featured several tables for players, teams, tournaments and the relationships between them, played-for, played-in. This worked for the initial prototype allowing for the basic create, read, update, and delete operations within the database. However, as we began to implement the graph presentation with Cytoscape we faced issues

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/10.1145/nnnnnnn.nnnnnnn

implementing the multi-graph support without implementing complex queries. This lead to a refinement to Figure 2 which showcases the updated entity relation model.

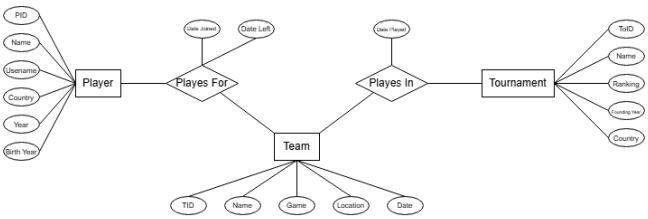


Figure 1: Old Entity Relation Diagram

The benefits of this model is rather than having three different types of entities for each team, player, and tournament we simply have a Node entity. Each Node has a NodeID which acts as the primary key. Then the team, player, or tournament attribute is stored withing the NodeType value. Since each team, player, and tournament has different amount of attributes stored with it the Attributes column is a JSON file allowing for flexible metadata on the node.

Similarly, rather than having two different tables for the played-for and played-in relationship we consolidated it to one Edge entity. Each edge has a an EdgeID acting as the primary key. As well as two node id's, SourceNodeID and TargetNodeID, which describe the nodes the edge connects as well as are foreign keys to the node table. For the same reasons as the node entity we gave the edge entity and EdgeType value which holds the played-in or played-for relationship. Lastly, since each relasonship holds a different number of descriptors edges have a MetaData value which acts identically to the Attributes column of nodes.

To facilitate the multi-graph support we've added a new entity to represent the graphs themselves. Each graph has a GraphID which is the primary key of the table. Each graph also has a name that can be displayed. In addition, each graph has a description to further add details to the graph. We've also added a new relationship between the entities, Member of, this represents the relationship between a graph and a node or edge. Each membership has it's own MemeberID which acts as the primary key. A nodeID or edgeID which determines which entity belongs to the graph. The graphID to determine which graph the entity belongs to. This schema allows for easy graph integration and multi-graph support, in addition for further expansion if we decide to add more types of nodes or edges.

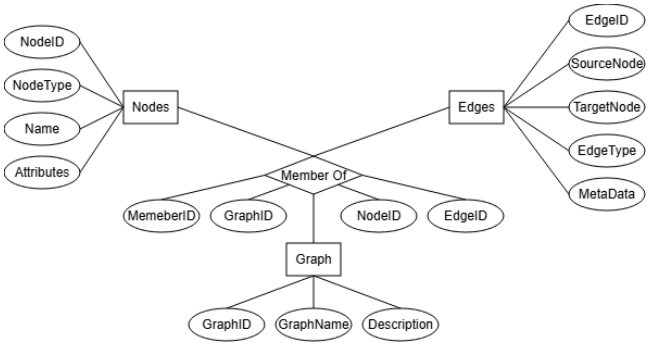


Figure 2: Updated Entity Relation Model

4 Tech Stack

Likewise to past reports this project utilizes three different categories of technology. A database, back-end processing, and front-end/presentation.

4.1 Database

In previous reports the database we decided on was **MySQL**, which is based around the client-server model. To simplify development and improve collaboration we've transitioned to **SQLite**, a lightweight relational database that follows a serverless architecture. This design makes it extremely easy to set up and allows team members to share the database simply by distributing the file.

4.2 Back-End

For the back end we've utilized **Python** for it's expansive libraries and data analysis tools. We leveraged Python's frameworks and libraries to handle core logic and data interaction with the SQLite database. Discussed later, we utilized Flask for webpage routing and Playwright to populate the database.

4.3 Front-End

The front end utilizes HTML and CSS for the webpage templates as well as the presentation. To present the graph presentation of the network we utilize Cytoscape which is an open-source network visualizer. Which allows us to render complex relationships in an intuitive way. Through its JavaScript integration we are able to embed interactive graphs directly into the webpage.

5 Implementation

5.1 Populating the Database

To populate the initial database we've developed a web scrapper that utilizes Playwright to load web pages and scrape the information from the html tags then inserts them into the database. The scraper checks for duplicate nodes, in that case the node's attributes are updated. In addition, the scrapper creates the respective edges as well. Below is a list of webpages where we scrape the information.

(1) https://prosettings.net/players/
(2) https://en.wikipedia.org/wiki/
(3) https://liquipedia.net/valorant/S-Tier_Tournaments
(4) https://liquipedia.net/valorant/A-Tier_Tournaments

In addition to the web scraper, the project also includes several dedicated webpages that allow users to insert data directly into the database. These pages utilize forms to fill in a SQL statement that is then executed by the database manager. Figures 3 showcases the database insertion webpages.

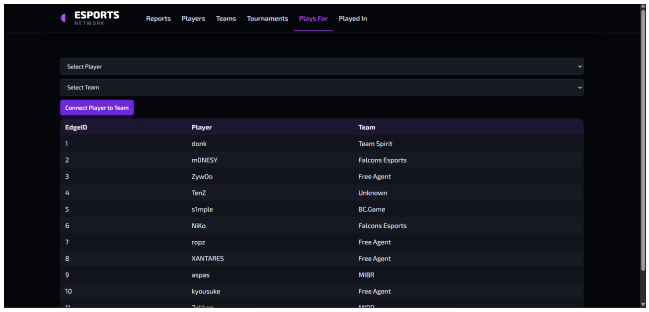
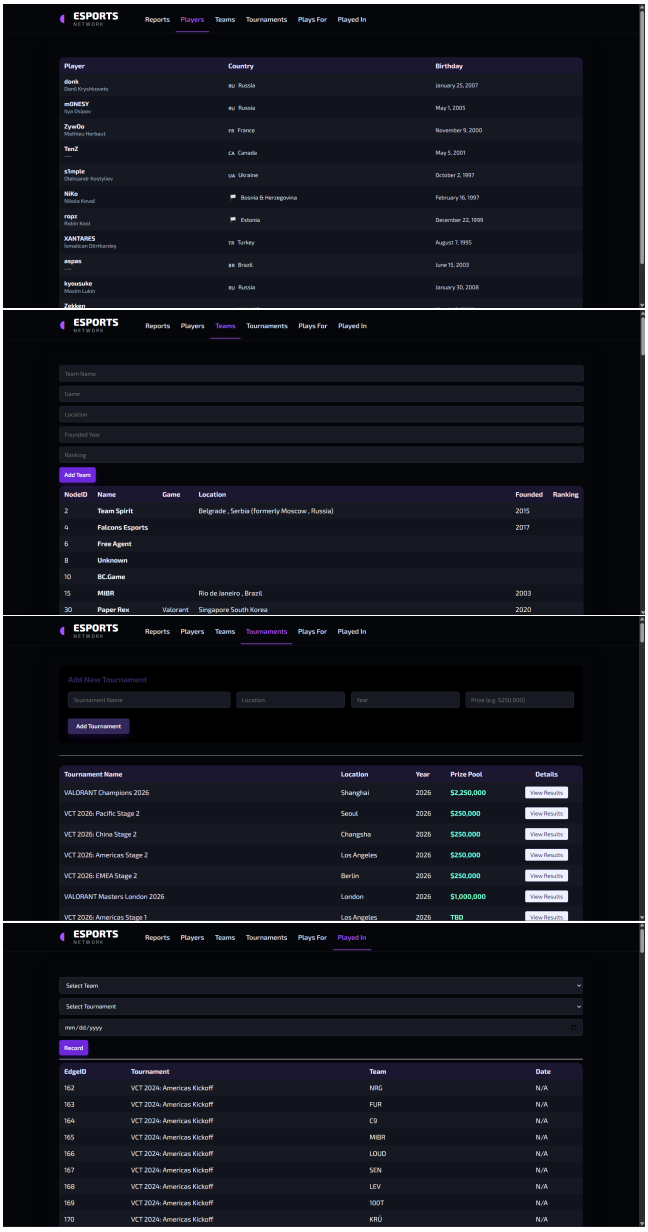


Figure 3: Database Insertion Webpages



5.2 Reports Page

As mentioned previously the project utilizes Flask to handle webpage routing. Each webpage calls from the database and then fills in the template html page with the returned information from the database. The front page get the entire database then utilizes Cytoscape's java script integration to present the information. Figure 4 showcases the front webpage.

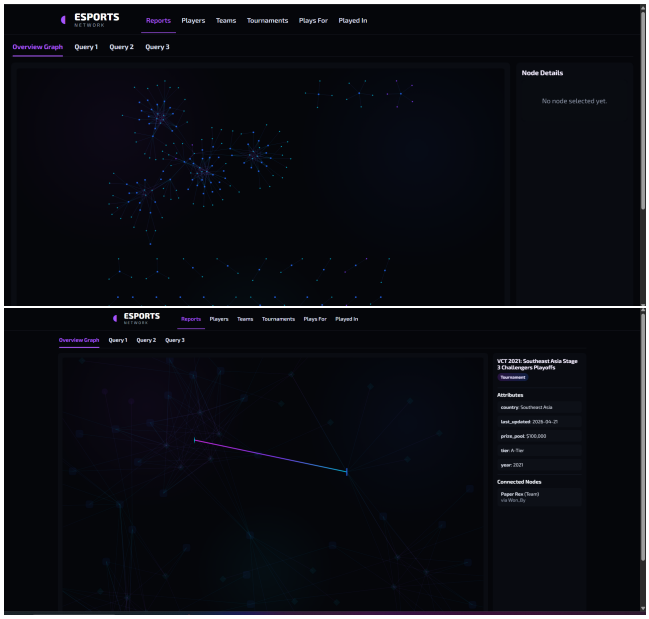


Figure 4: Reports Web Page

5.3 Query 1

For this project the initial guidelines were to implement a *Co-Author Network*. For this specific project we implemented a *Teammate Network* which takes in a specific player and then returns all other players that play for the same team as that player. The webpage executed the below queries to get the respective nodes and edges. Which selects all nodes that are connected to the same team as the given player then gets all appropriate edges. Figure 5 also showcases the webpage itself.

```
SELECT DISTINCT n.NodeID, n.Name, n.NodeType, n.Attributes
    FROM Nodes n
    JOIN GraphMemberships gm ON gm.NodeID = n.NodeID
    WHERE gm.GraphID = 1
    AND n.NodeType IN ("Player", "Team")
    ORDER BY n.NodeType, n.Name
SELECT DISTINCT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
e.EdgeType, e.Metadata
    FROM Edges e
    JOIN GraphMemberships gm ON gm.EdgeID = e.EdgeID
    WHERE gm.GraphID = 1
    AND e.EdgeType = "Plays_For"
```

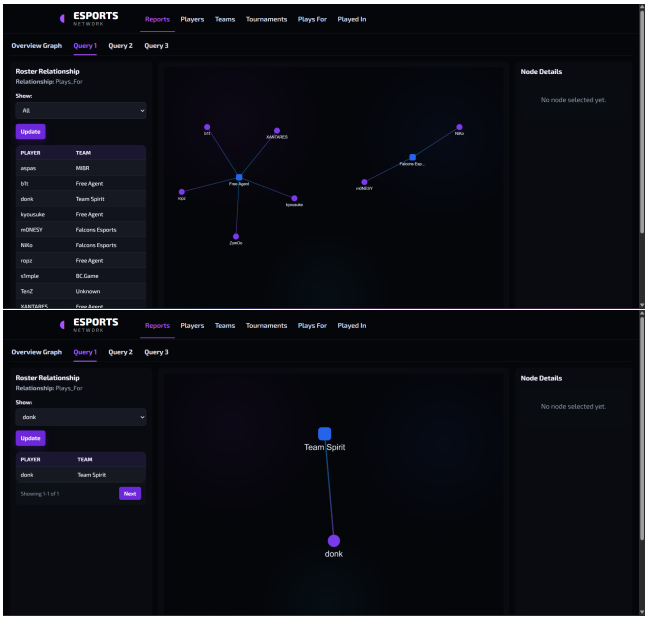


Figure 5: Query 1 Webpage with Example

5.4 Query 2

For this project the initial guidelines were to implement a *h-index filer*. Which we implemented as a team filter based on both number of tournaments participated in and number won. Which takes a specific a specific number to represent the h-index. Then applies a filter on the teams and then presents that graphically with the tournaments and teams that participated in them. The first two queries are executed then the following two are applied to filter based on wins or particpation respectivly. Figure 6 showcases the webpage itself.

```
SELECT SourceNodeID, COUNT(DISTINCT TargetNodeID)
    AS participant_count
    FROM Edges
    WHERE EdgeType = "Played_In"
    GROUP BY SourceNodeID
SELECT NodeID, Name, NodeType, Attributes
    FROM Nodes
    WHERE NodeType = "Team"
    ORDER BY Name
```

```
SELECT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
    tr.Name AS tournament_name,
    tr.Attributes AS tournament_attributes
    FROM Edges e
    JOIN Nodes tr ON tr.NodeID = e.SourceNodeID
    WHERE e.EdgeType = "Won_By"
      AND tr.NodeType = "Tournament"
    ORDER BY tr.Name
```

```
SELECT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
    tr.Name AS tournament_name,
    tr.Attributes AS tournament_attributes
    FROM Edges e
    JOIN Nodes tr ON tr.NodeID = e.SourceNodeID
    WHERE e.EdgeType = "Played_In"
      AND tr.NodeType = "Tournament"
    ORDER BY tr.Name
```

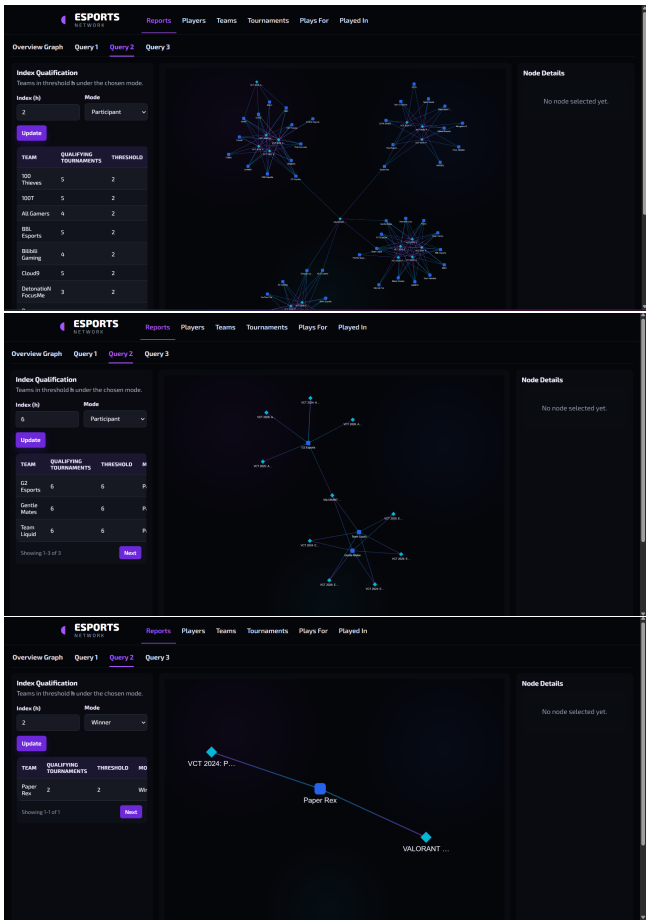


Figure 6: Query 2 Webpage with Examples

5.5 Query 3

For this project the guidelines were to implement a *Q1 Journal Influence Network*. Which we implemented as a Q1 Tournaments filter. We defined a Q1 tournament as a tournament where the prize pool is greater than 1 million dollars. The influence network of a Q1 tournament is all teams that participated in that tournament. For this query the below statements are executed, first getting the qualifying tournaments, then all teams that are associated with it and all matching edges. Figure 7 showcases the webpage.

```
SELECT DISTINCT n.NodeID, n.Name, n.NodeType, n.Attributes
    FROM Nodes n
    JOIN GraphMemberships gm ON gm.NodeID = n.NodeID
    WHERE gm.GraphID = 3
    AND n.NodeType = "Tournament"
    ORDER BY n.Name

SELECT DISTINCT n.NodeID, n.Name, n.NodeType, n.Attributes
    FROM Nodes n
    JOIN GraphMemberships gm ON gm.NodeID = n.NodeID
    WHERE gm.GraphID = 3
    AND n.NodeType = "Team"
    ORDER BY n.Name

SELECT DISTINCT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
    e.EdgeType
    FROM Edges e
    JOIN GraphMemberships gm ON gm.EdgeID = e.EdgeID
    WHERE gm.GraphID = 3
    AND e.EdgeType IN ("Played_In", "Won_By")
    ORDER BY e.EdgeID
```

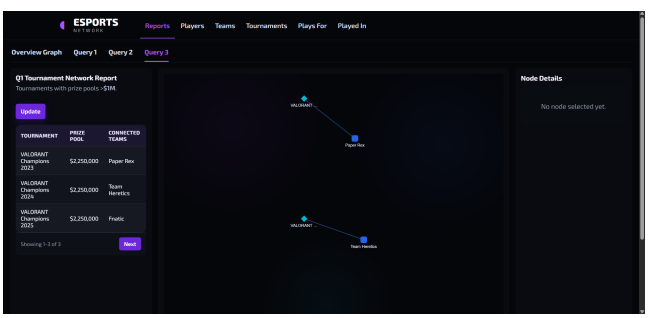


Figure 7: Query 3 Webpage with Example

5.6 Timeline Comparison

Below is the timeline outlined in phase one.

- Phase 1:
  - Initial Design Presentation
    Date: March 3rd
  - Initial Project Outline Report
    Date: March 7th
- Phase 2:
  - Working Prototype Presentation
    Date: March 24th
  - General Design Report
    Date: March 28th
- Phase 3:
  - Fully Function Prototype Presentation
    Date: April 21st
  - Expanded Design Report
    Date: April 25th
- Phase 4:
  - Final Project Demo
    Date: May 5th
  - Finalized Project Report
    Date: May 3rd

As of writing this report it's clear we are on track to complete phase four on time. The initial project has been outlined and created. The initial prototype was created allowing for population of the database. As well as initial database schema was defined and them later revised. The functional prototype has been demonstrated in this report. As well as the three required queries co-author network, h-index filter, and Q1-journal influence network.

6 Conclusion

E-Sports has rapidly evolved into a global form of entertainment across titles such as *League of Legends*, *Counter-Strike*, and *Dota 2*. Despite this growth, the analytical infrastructure supporting e-sports still lags behind traditional sports. The E-Sports Competitive Network directly addresses this limitation by introducing a relational database designed to model the relationships between players, teams, and tournaments across multiple games. These relationships are showcased through an updated relationship schema and tech stack. The E-Sports Competitive Network allows for mass population of it's database through a custom developed web scrapper and manual insertion through its web pages. The E-Sports Competitive Network features three key queries, the teammate network which showcases how players are related across teams. A h-index filter to showcase teams based on the number or tournaments they've participated in as well as won. Lastly, the Q1 journal influence network which showcases all teams that have participated in a tournament with a prize pool over 1 million dollars. The E-Sports Competitive Network lays the groundwork for more advanced analytical tools that can benefit coaches, analysts, and researchers.

References

[1] Microsoft. 2020. Playwright for Python. https://github.com/microsoft/playwright-python.

[2] Paul Shannon, Andrew Markiel, Owen Ozier, Nitin S. Baliga, Jonathan T. Wang, Daniel Ramage, Nada Amin, Benno Schwikowski, and Trey Ideker. 2003. Cytoscape: A Software Environment for Integrated Models of Biomolecular Interaction Networks. *Genome Research* 13, 11 (2003), 2498–2504. doi:10.1101/gr.1239303

[2] [1]